HW01 Advanced Special Topics Deep Learning

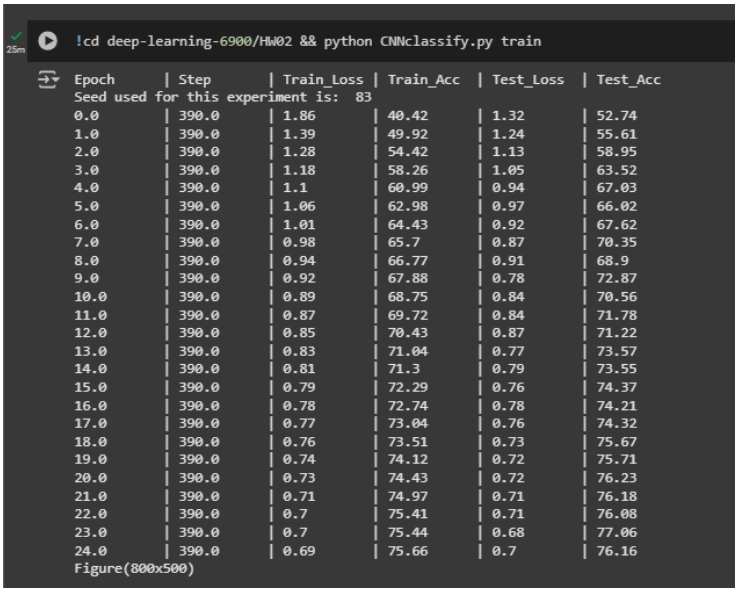
Alex Darwiche

9/26/2025

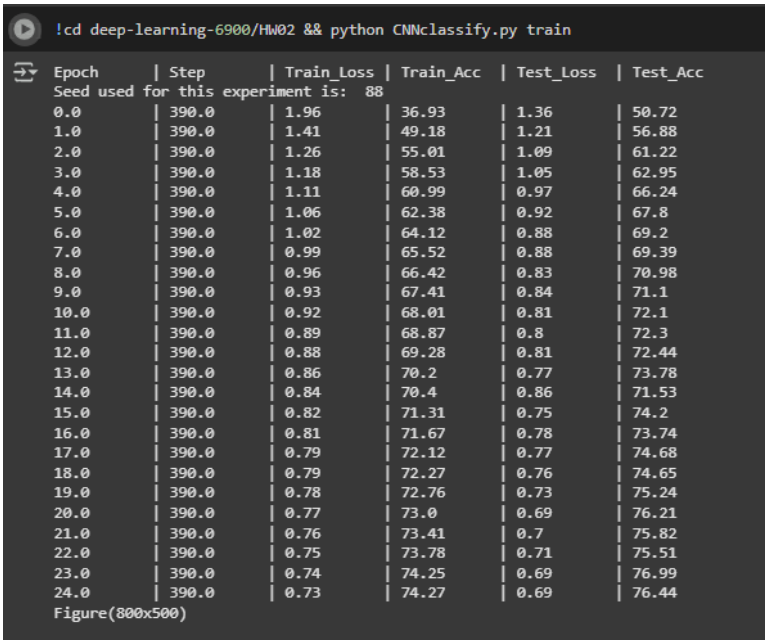
My Colab Notebook (Contains code to run model if desired): link

Model Runs

(two screenshots of training and testing results of your neural network design)

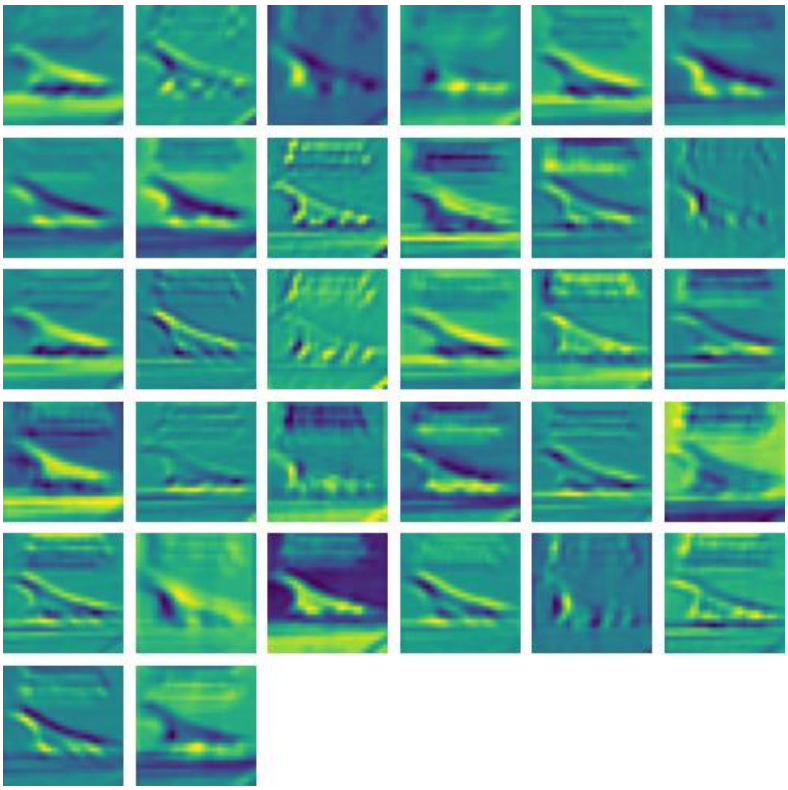


```
!cd deep-learning-6900/HW02 && python CNNclassify.py train

Epoch      | Step       | Train_Loss | Train_Acc  | Test_Loss  | Test_Acc
Seed used for this experiment is:  83
0.0        | 390.0      | 1.86       | 40.42      | 1.32       | 52.74
1.0        | 390.0      | 1.39       | 49.92      | 1.24       | 55.61
2.0        | 390.0      | 1.28       | 54.42      | 1.13       | 58.95
3.0        | 390.0      | 1.18       | 58.26      | 1.05       | 63.52
4.0        | 390.0      | 1.1        | 60.99      | 0.94       | 67.03
5.0        | 390.0      | 1.06       | 62.98      | 0.97       | 66.02
6.0        | 390.0      | 1.01       | 64.43      | 0.92       | 67.62
7.0        | 390.0      | 0.98       | 65.7       | 0.87       | 70.35
8.0        | 390.0      | 0.94       | 66.77      | 0.91       | 68.9
9.0        | 390.0      | 0.92       | 67.88      | 0.78       | 72.87
10.0       | 390.0      | 0.89       | 68.75      | 0.84       | 70.56
11.0       | 390.0      | 0.87       | 69.72      | 0.84       | 71.78
12.0       | 390.0      | 0.85       | 70.43      | 0.87       | 71.22
13.0       | 390.0      | 0.83       | 71.04      | 0.77       | 73.57
14.0       | 390.0      | 0.81       | 71.3       | 0.79       | 73.55
15.0       | 390.0      | 0.79       | 72.29      | 0.76       | 74.37
16.0       | 390.0      | 0.78       | 72.74      | 0.78       | 74.21
17.0       | 390.0      | 0.77       | 73.04      | 0.76       | 74.32
18.0       | 390.0      | 0.76       | 73.51      | 0.73       | 75.67
19.0       | 390.0      | 0.74       | 74.12      | 0.72       | 75.71
20.0       | 390.0      | 0.73       | 74.43      | 0.72       | 76.23
21.0       | 390.0      | 0.71       | 74.97      | 0.71       | 76.18
22.0       | 390.0      | 0.7        | 75.41      | 0.71       | 76.08
23.0       | 390.0      | 0.7        | 75.44      | 0.68       | 77.06
24.0       | 390.0      | 0.69       | 75.66      | 0.7        | 76.16
Figure(800x500)
```



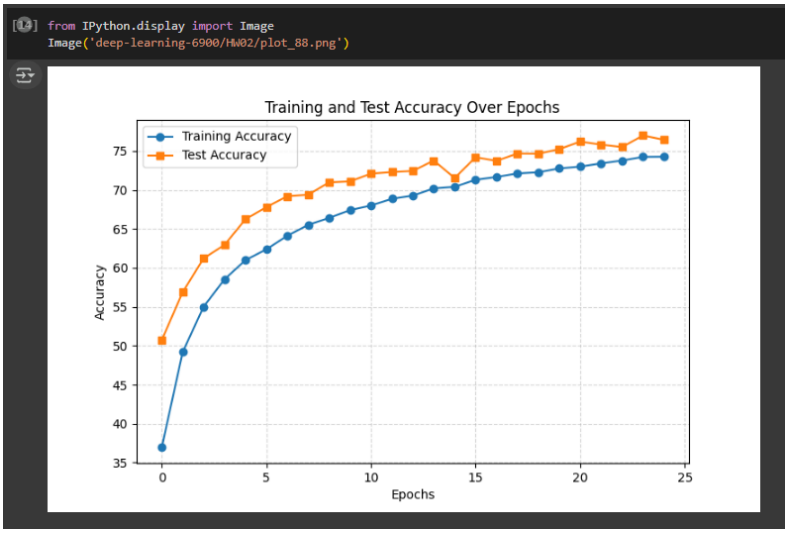
```
!cd deep-learning-6900/HW02 && python CNNclassify.py train

Epoch      | Step       | Train_Loss | Train_Acc  | Test_Loss  | Test_Acc
Seed used for this experiment is:  88
0.0        | 390.0      | 1.96       | 36.93      | 1.36       | 50.72
1.0        | 390.0      | 1.41       | 49.18      | 1.21       | 56.88
2.0        | 390.0      | 1.26       | 55.01      | 1.09       | 61.22
3.0        | 390.0      | 1.18       | 58.53      | 1.05       | 62.95
4.0        | 390.0      | 1.11       | 60.99      | 0.97       | 66.24
5.0        | 390.0      | 1.06       | 62.38      | 0.92       | 67.8
6.0        | 390.0      | 1.02       | 64.12      | 0.88       | 69.2
7.0        | 390.0      | 0.99       | 65.52      | 0.88       | 69.39
8.0        | 390.0      | 0.96       | 66.42      | 0.83       | 70.98
9.0        | 390.0      | 0.93       | 67.41      | 0.84       | 71.1
10.0       | 390.0      | 0.92       | 68.01      | 0.81       | 72.1
11.0       | 390.0      | 0.89       | 68.87      | 0.8        | 72.3
12.0       | 390.0      | 0.88       | 69.28      | 0.81       | 72.44
13.0       | 390.0      | 0.86       | 70.2       | 0.77       | 73.78
14.0       | 390.0      | 0.84       | 70.4       | 0.86       | 71.53
15.0       | 390.0      | 0.82       | 71.31      | 0.75       | 74.2
16.0       | 390.0      | 0.81       | 71.67      | 0.78       | 73.74
17.0       | 390.0      | 0.79       | 72.12      | 0.77       | 74.68
18.0       | 390.0      | 0.79       | 72.27      | 0.76       | 74.65
19.0       | 390.0      | 0.78       | 72.76      | 0.73       | 75.24
20.0       | 390.0      | 0.77       | 73.0       | 0.69       | 76.21
21.0       | 390.0      | 0.76       | 73.41      | 0.7        | 75.82
22.0       | 390.0      | 0.75       | 73.78      | 0.71       | 75.51
23.0       | 390.0      | 0.74       | 74.25      | 0.69       | 76.99
24.0       | 390.0      | 0.73       | 74.27      | 0.69       | 76.44
Figure(800x500)
```

First CONV Layer Outputs:



Model Visualization

(a screenshot of the plot shows the curves of the training accuracy and testing accuracy)



MACs and Parameters

```
Resnet20: MACs: 41.616M, Parameters: 272.474K
User Model: MACs: 29.187M, Parameters: 12.576M
```

Random seeds and Final Accuracies

Seeds	Final Test Accuracy
88	76.44%
83	76.16%
54	75.05%

Mean: 75.88

Standard Deviation: 0.60

Speed Test:

```
!cd deep-learning-6900/HW02 && python CNNclassify.py speedTest 'img/cifar10_airplane_3.png'

Inference time for User Model: 1.4217 seconds
Inference time for Resnet20: 3.8196 seconds
```

This is an inference test going over the same image 1000 times, with both Resnet20 and the "User Model" that I trained.

CODE:

```
########################################################
# First of the first, please start writing it early!
########################################################

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.utils.data as Data
import torchvision
import torch.optim as optim
import os
from PIL import Image
```

```
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import sys
import pandas as pd
import random
import numpy as np
# import torch_directml
from resnet20_cifar import resnet20
import time

import warnings
warnings.filterwarnings("ignore")
# Normalize the CIFAR10 Dataset (from Pytorch Website)
transform = transforms.Compose([
        transforms.Resize((32, 32)),  # Resize image to 32x32
        transforms.ToTensor(),        # Convert image to tensor
        transforms.Normalize(mean=[0.4914, 0.4822, 0.4465], std=[0.2470, 0.2435,
0.2616])  # CIFAR-10 normalization
    ])

# Define the data augmentation transformations
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(32, scale=(0.8, 1.0)),  # Random crop and resize
    transforms.RandomHorizontalFlip(),  # Random horizontal flip
    transforms.RandomRotation(10),  # Random rotation by 10 degrees
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2,
hue=0.1),  # Random color jitter
    transforms.ToTensor(),  # Convert image to tensor
    transforms.Normalize(mean=[0.4914, 0.4822, 0.4465], std=[0.2470, 0.2435,
0.2616])  # Normalize with standard ImageNet values
])

# ----------------- prepare training data ----------------------
train_data = torchvision.datasets.CIFAR10(
    root='./data.cifar10',                          # location of the dataset
    train=True,                                     # this is training data
    transform=train_transform,    # Converts a PIL.Image or numpy.ndarray to
torch.FloatTensor of shape (C x H x W)
    download=True                                   # if you haven't had the
dataset, this will automatically download it for you
)


# Define a batch size to set with, using 128
batch_size = 128
```

```
# Load the training data from the dataset, breaking it into batches
train_loader = Data.DataLoader(dataset=train_data, batch_size = batch_size,
shuffle=True)

# ----------------- prepare testing data -----------------------
test_data = torchvision.datasets.CIFAR10(root='./data.cifar10/', train=False,
transform=transform)

# Load the training data from the dataset, breaking it into batches
test_loader = Data.DataLoader(dataset=test_data,  batch_size = batch_size,
shuffle=True )

# from thop import profile, clever_format

# model = resnet20()
# # Example input shape for CIFAR-10 (batch size 1, 3 channels, 32x32)
# dummy_input = torch.randn(1, 3, 32, 32)
# # Profile the model
# macs, params = profile(model, inputs=(dummy_input,))
# # Format results
# macs, params = clever_format([macs, params], "%.3f")
# print(f"Resnet20: MACs: {macs}, Parameters: {params}")


# ----------------- build the model -----------------------

# Define a Deep Nueral Network with only Fully Connected Layers or Batch
Normalization
class net(nn.Module):
    def __init__(self):
        super(net, self).__init__()
        # Convolutional Layers
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=5,
padding=0, stride=1)  # Reduced number of filters
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3,
padding=1)  # Reduced number of filters
        self.bn1 = nn.BatchNorm2d(32)
        self.bn2 = nn.BatchNorm2d(64)

        # Fully Connected Layers
        self.fc1 = nn.Linear(12544, 1000)  # Adjust based on input image size
after convolutional layers
        self.fc2 = nn.Linear(1000, 10)
```

```
        # Dropout Layers
        self.dropout = nn.Dropout(p=0.1)  # Randomly drop 50% of the neurons

    def forward(self, x):
        # Convolutional Layers with Batch Normalization and ReLU activation
        x = F.relu(self.bn1(self.conv1(x)))
        x = F.relu(self.bn2(self.conv2(x)))
        x = F.max_pool2d(x, 2)  # Add max-pooling after convolutions to reduce
spatial size

        # Flatten before passing to fully connected layers
        x = torch.flatten(x, 1)  # Flatten the output from conv layers into a
vector

        # Fully Connected Layers with Batch Normalization, ReLU, and Dropout
        x = F.relu(self.fc1(x))
        x = self.dropout(x)  # Apply dropout
        x = self.fc2(x)  # Final output layer (no activation here, typically for
classification)

        return x

# ------ maybe some helper functions -----------
def test_accuracy(model, test_loader, loss_func, device='cpu'):
    '''
    This function will return the accuracy of the model
    on the testing data.

    Args:
        model: The trained PyTorch model.
        test_loader: DataLoader for the testing dataset.
        loss_func: Loss function used for evaluation.
        device: The device to run the model on ('cpu' or 'cuda').

    Returns:
        Tuple: (test accuracy %, test loss)
    '''
    model.eval()  # Switch the model to evaluation mode
    correct = 0
    total = 0
    total_loss = 0  # Initialize total loss

    with torch.no_grad():  # Disable gradient computation for inference
        for images, target in test_loader:
            # Move data to the correct device
```

```
            images, target = images.to(device), target.to(device)

            outputs = model(images)  # Forward pass through the model
            loss = loss_func(outputs, target)  # Compute the loss

            total_loss += loss.item() * target.size(0)  # Accumulate the weighted
loss
            _, predicted = outputs.max(1)  # Get predicted class (the one with
highest probability)
            total += target.size(0)  # Accumulate the total number of samples
            correct += (predicted == target).sum().item()  # Count correct
predictions

    # Compute final accuracy and average loss
    accuracy = 100 * correct / total
    avg_loss = total_loss / total  # Average loss over all samples

    return accuracy, avg_loss



def save_model(test_accuracy):
    '''
    This function will save the model in the event that it exceeds
    a previous model's test accuracy, or if a model does not exist.

    '''

    model_path = "./model/model.pt" # Where to save model and what to name it.

    # Determine if a model already exists, save model if not
    if os.path.exists(model_path):
        model2 = torch.load(model_path) # Load the model from the save. This
isn't exactly the "load_state_dict" as I added test_accuracy to the model save
        if model2['test_accuracy'] < test_accuracy: # If the "new model" has
better test accuracy, then continue
            os.remove(model_path) # Remove the old model
            torch.save({ # save the new model parameters AND its test accuracy
                'model_state_dict': model.state_dict(),
                'test_accuracy': test_accuracy
            }, model_path)
    else:
        torch.save({ # save the new model parameters AND its test accuracy
            'model_state_dict': model.state_dict(),
            'test_accuracy': test_accuracy
```

```
        }, model_path)

def load_model():
    '''
    This function loads the model_state_dict or model state dictionary from the
saved
    model.

    '''
    model = net()
    checkpoint = torch.load("./model/model.pt")['model_state_dict']

    # Remove unexpected keys
    filtered_state_dict = {k: v for k, v in checkpoint.items() if k in
model.state_dict()}

    model.load_state_dict(filtered_state_dict)
    return model


def compute_train_accuracy(model, train_loader, loss_func, device='cpu'):
    """
    Computes the accuracy and average loss of the model on the training dataset.

    Args:
        model: The trained PyTorch model.
        train_loader: DataLoader for the training dataset.
        loss_func: Loss function used for evaluation.
        device: 'cpu' or 'cuda' depending on available hardware.

    Returns:
        Tuple: (accuracy %, average loss)
    """
    model.train()  # Set model to training mode
    correct = 0
    total = 0
    total_loss = 0  # Track cumulative loss

    with torch.no_grad():  # Disable gradient computation
        for images, target in train_loader:
            images, target = images.to(device), target.to(device)  # Move to
device

            outputs = model(images)  # Forward pass
            loss = loss_func(outputs, target)  # Compute batch loss
```

```
            total_loss += loss.item() * target.size(0)  # Accumulate weighted
loss

            _, predicted = outputs.max(1)  # Get predicted class
            total += target.size(0)  # Update total count
            correct += (predicted == target).sum().item()  # Count correct
predictions

    # Compute final accuracy and average loss
    accuracy = 100 * correct / total
    avg_loss = total_loss / total  # Average loss over all samples

    return accuracy, avg_loss


def inference_speed_test(model):

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    start_time = time.time()
    for i in range(1000):
        # Need to "normalize" the picture that is being input, ensuring its the
correct size
        # and the channels are normalized based on mean/std.
        transform = transforms.Compose([
            transforms.Resize((32, 32)),  # Resize image to 32x32
            transforms.ToTensor(),        # Convert image to tensor
            transforms.Normalize(mean=[0.4914, 0.4822, 0.4465], std=[0.2470,
0.2435, 0.2616])  # CIFAR-10 normalization
        ])

        # Load image from file
        image_path = sys.argv[2]
        image = Image.open(image_path).convert("RGB")

        # Apply transformations
        image_tensor = transform(image).unsqueeze(0).to(device)

        # CIFAR-10 Class Labels
        labels_map = {
            0: 'airplane', 1: 'automobile', 2: 'bird', 3: 'cat', 4: 'deer',
            5: 'dog', 6: 'frog', 7: 'horse', 8: 'ship', 9: 'truck'
        }

        # Run inference
```

```
        with torch.no_grad():
            model.eval()
            output = model(image_tensor)
            predicted_class = output.argmax(1).item()
        # print(f"prediction result: {labels_map[predicted_class]}")

    end_time = time.time()
    return end_time - start_time


# Intermediate Results outputs
activations = []

def hook_fn(module, input, output):
    activations.append(output)

# This if statement dictates the interactions depending on the arugments passed
to command line.
# IF 'Train', train the model.
# IF 'Test' or 'Predict', try to predict the command line image's class
if 'train' in sys.argv[1:]:
    # Check if CUDA is available
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    # import torch_directml

    # Set the device to DirectML
    # device = torch_directml.device()

    # Initialize model, loss function, and optimizer, and move model to the
device
    model = net().to(device)
    loss_func = nn.CrossEntropyLoss()

    # # Example input shape for CIFAR-10 (batch size 1, 3 channels, 32x32)
    # dummy_input = torch.randn(1, 3, 32, 32)
    # # Profile the model
    # macs, params = profile(model, inputs=(dummy_input,))
    # # Format results
    # macs, params = clever_format([macs, params], "%.3f")
    # print(f"User Model: MACs: {macs}, Parameters: {params}")

    # model.conv1.register_forward_hook(hook_fn)
    # model.conv2.register_forward_hook(hook_fn)
    #optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9,
weight_decay=0.01)
```

```
    optimizer = optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999),
eps=1e-08, weight_decay=0)
    epochs = 25

    # Create an empty dataframe for results
    results = pd.DataFrame({"Epoch": [], "Step": [], "Train_Loss": [],
"Train_Acc": [], "Test_Loss": [], "Test_Acc": []})
    print(" | ".join("{:<10}".format(col) for col in results.columns.to_list()))

    test_accuracy_list = []
    train_accuracy_list = []

    # Set the seed for reproducibility
    seed = random.randint(1, 100)
    torch.manual_seed(seed)
    np.random.seed(seed)

    print("Seed used for this experiment is: ",seed)

    # Training loop
    for epoch in range(epochs):
        train_loss = 0
        correct = 0
        total = 0


        for step, (input, target) in enumerate(train_loader):
            input, target = input.to(device), target.to(device)  # Move data to
the device
            model.train()  # Set model in training mode
            optimizer.zero_grad()  # Reset gradients
            output = model(input)  # Forward pass
            loss = loss_func(output, target)  # Compute loss
            loss.backward()  # Backpropagate gradients
            optimizer.step()  # Update weights

            train_loss += loss.item() * target.size(0)  # Accumulate weighted
loss
            _, predicted = output.max(1)  # Get predicted class
            total += target.size(0)
            correct += (predicted == target).sum().item()  # Count correct
predictions

        # Get test accuracy and loss (ensure model is in eval mode for testing)
```

```
        test_acc, test_loss = test_accuracy(model, test_loader, loss_func,
device)

        test_accuracy_list.append(test_acc)
        # Compute training accuracy (after the epoch) and loss
        train_acc = 100 * correct / total
        train_accuracy_list.append(train_acc)
        avg_train_loss = train_loss / total

        # Create a new row in the results dataframe
        new_row = pd.DataFrame([{
            "Epoch": epoch,
            "Step": step,
            "Train_Loss": round(avg_train_loss, 2),
            "Train_Acc": round(train_acc, 2),
            "Test_Loss": round(test_loss, 2),
            "Test_Acc": round(test_acc, 2)
        }])
        results = pd.concat([results, new_row], ignore_index=True)

        # Print the results from the first step of the new epoch
        print(" | ".join("{:<10}".format(str(value)) for value in results.iloc[-
1]))


        # Optionally save the model
        save_model(test_acc)

    plt.figure(figsize=(8, 5))
    plt.plot(range(len(train_accuracy_list)), train_accuracy_list,
label='Training Accuracy', marker='o')
    plt.plot(range(len(test_accuracy_list)), test_accuracy_list, label='Test
Accuracy', marker='s')

    plt.title('Training and Test Accuracy Over Epochs')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend()
    name = "plot_" + str(seed) + '.png'
    plt.savefig(name)
    plt.show()

elif 'predict' in sys.argv[1:] or 'test' in sys.argv[1:]:
```

```
    def visualize_activation(model, input_image, layer_name):
        # Hook to capture the activations
        activation_map = []

        def hook_fn(module, input, output):
            activation_map.append(output.detach())

        # Register hook to the layer
        layer = dict([*model.named_modules()])[layer_name]
        hook = layer.register_forward_hook(hook_fn)

        # Perform inference
        with torch.no_grad():
            model.eval()
            output = model(input_image)

        # Remove hook to avoid memory leaks
        hook.remove()

        # Extract the activations (this is the output from the chosen layer)
        activation = activation_map[0][0]  # Shape: [batch_size, channels,
height, width]

        # Number of channels
        num_channels = activation.size(0)

        # Calculate grid size (smallest square grid that fits all channels)
        grid_size = int(np.ceil(np.sqrt(num_channels)))  # Grid size is the
ceiling of sqrt(num_channels)

        # Create the plot with enough subplots
        fig, axes = plt.subplots(6, 6, figsize=(5, 5))

        # Flatten the axes array for easier indexing
        axes = axes.flatten()

        # Plot each channel in the grid
        for i in range(32):
            axes[i].imshow(activation[i].cpu().numpy(), cmap='viridis')
            axes[i].axis('off')

        # Hide any empty subplots if the grid size exceeds the number of channels
        for i in range(32, len(axes)):
            axes[i].axis('off')
```

```
        plt.subplots_adjust(left=0, right=1, top=1, bottom=0, wspace=.1,
hspace=.1)
        plt.savefig('CONV_rslt.png')
        plt.show()

    # Need to "normalize" the picture that is being input, ensuring its the
correct size
    # and the channels are normalized based on mean/std.
    transform = transforms.Compose([
        transforms.Resize((32, 32)),  # Resize image to 32x32
        transforms.ToTensor(),        # Convert image to tensor
        transforms.Normalize(mean=[0.4914, 0.4822, 0.4465], std=[0.2470, 0.2435,
0.2616])  # CIFAR-10 normalization
    ])

    # Load image from file
    image_path = sys.argv[2]  # Replace with your file path
    image = Image.open(image_path).convert("RGB")

    # Apply transformations
    image_tensor = transform(image).unsqueeze(0)  # Add batch dimension (1, 3,
32, 32)


    model = load_model()

    # Visualize activations during inference (e.g., for the first convolutional
layer "conv1")
    visualize_activation(model, image_tensor, "conv1")

    # Run inference
    with torch.no_grad():
        model.eval()
        output = model(image_tensor)
        predicted_class = output.argmax(1).item()

    # CIFAR-10 Class Labels
    labels_map = {
        0: 'airplane', 1: 'automobile', 2: 'bird', 3: 'cat', 4: 'deer',
        5: 'dog', 6: 'frog', 7: 'horse', 8: 'ship', 9: 'truck'
    }

    print(f"prediction result: {labels_map[predicted_class]}")

elif 'resnet20' in sys.argv[1:] or 'test' in sys.argv[1:]:
```

```
    # Check what device exists
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Initialize resnet model
    model = resnet20()

    # Load the model from memory
    checkpoint = torch.load("./model/resnet20_cifar10_pretrained.pt",
map_location=torch.device('cpu'))
    model.load_state_dict(checkpoint)

    # Initialize Loss Function
    loss_func = nn.CrossEntropyLoss()

    # Test and Reporting Training Accuracy
    test_acc, test_loss = test_accuracy(model, test_loader, loss_func, device)
    print('Test Accuracy of Resnet20: ', test_acc, "%")

elif 'predictResnet20' in sys.argv[1:] or 'test' in sys.argv[1:]:

    # Need to "normalize" the picture that is being input, ensuring its the
correct size
    # and the channels are normalized based on mean/std.
    transform = transforms.Compose([
        transforms.Resize((32, 32)),  # Resize image to 32x32
        transforms.ToTensor(),        # Convert image to tensor
        transforms.Normalize(mean=[0.4914, 0.4822, 0.4465], std=[0.2470, 0.2435,
0.2616])  # CIFAR-10 normalization
    ])

    # Load image from file
    image_path = sys.argv[2]  # Replace with your file path
    image = Image.open(image_path).convert("RGB")

    # Apply transformations
    image_tensor = transform(image).unsqueeze(0)  # Add batch dimension (1, 3,
32, 32)

    model = resnet20()
    checkpoint = torch.load("./model/resnet20_cifar10_pretrained.pt",
map_location=torch.device('cpu'))
    model.load_state_dict(checkpoint)

    # Run inference
    with torch.no_grad():
```

```
        model.eval()
        output = model(image_tensor)
        predicted_class = output.argmax(1).item()

    # CIFAR-10 Class Labels
    labels_map = {
        0: 'airplane', 1: 'automobile', 2: 'bird', 3: 'cat', 4: 'deer',
        5: 'dog', 6: 'frog', 7: 'horse', 8: 'ship', 9: 'truck'
    }

    print(f"prediction result: {labels_map[predicted_class]}")

elif 'speedTest' in sys.argv[1:] or 'test' in sys.argv[1:]:

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    model1 = net()
    model1 = load_model()
    model2 = resnet20()
    model2.load_state_dict(torch.load("./model/resnet20_cifar10_pretrained.pt",
map_location=device))

    # Move to GPU
    model1 = model1.to(device)
    model2 = model2.to(device)

    speed1 = inference_speed_test(model1)
    print(f"Inference time for User Model: {speed1:.4f} seconds")
    speed2 = inference_speed_test(model2)
    print(f"Inference time for Resnet20: {speed2:.4f} seconds")
```